

Day 5 Homework: A second document type

Phase 1 · Week 1 · Day 5 · Sadasivam Mithran · Code: github.com/mithransadasivam/FDE (folder HW #5)

In short: my document extractor already read invoices. I taught it to read IT incident reports too. A model reads each report and fills in a form. The code then checks the form against a set of rules. If the form breaks a rule, it gets one more try, and if it still fails a person has to look at it. Nothing is fixed automatically.

Schema: the rules an incident report must follow

A **schema** is a checklist for the data: which fields exist, which values are allowed and which rules must hold. I wrote one for incident reports, called `IncidentReport`, in `app/schemas.py`. The existing invoice schema was not changed. The rules are:

- **Incident ID** must look like INC-2026-0917 (INC, a year, a four-digit number).
- **Severity** must be exactly P1, P2, P3 or P4. Words like "High" are rejected.
- **Started time** is required. **Resolved time** is optional, because an incident that is still open has not been resolved yet.
- **Root cause** is optional too: report 02 is still being investigated, so it has none.
- **Actions** must list at least one thing that was done.
- **Time order:** the incident cannot be resolved before it started.
- **One extra rule I added:** if one of the two times has a timezone and the other does not, the report is rejected. The computer cannot compare them, and I do not want it to guess a timezone.

```
class IncidentReport(BaseModel):
    incident_id: str = Field(pattern=r"^INC-\d{4}-\d{4}$")
    system: str = Field(min_length=2)
    severity: Literal["P1", "P2", "P3", "P4"]
    started_at: datetime
    resolved_at: datetime | None = None # open incidents have no resolution time
    impact: str
    root_cause: str | None = None # still under investigation on open incidents
    actions: list[str] = Field(min_length=1)

    @model_validator(mode="after")
    def resolved_after_started(self):
        if self.resolved_at is None:
            return self
        # an aware and a naive datetime cannot be compared; reject rather than guess a zone
        if (self.resolved_at.tzinfo is None) != (self.started_at.tzinfo is None):
            raise ValueError(
                "started_at and resolved_at must both have a timezone or both have none"
            )
        if self.resolved_at < self.started_at:
            raise ValueError("resolved_at is earlier than started_at")
        return self
```

```
SCHEMAS = {"invoice": Invoice, "incident": IncidentReport}
```

Results: running the extractor on the three reports

I added a `--type` option to the batch script, so it can read either invoices or incidents. For incidents the results table shows the severity and the system instead of an invoice total. I ran:

```
python -m scripts.extract_batch --retry --type incident --folder data/incidents
```

document	result	tries	detail
Day05_HW_Slide03_incident_01.txt	VALID	1	P1 Payments API
Day05_HW_Slide03_incident_02.txt	VALID	1	P3 ORD-ETL
Day05_HW_Slide03_incident_03.txt	VALID	1	P2 Wi-Fi

3 of 3 documents valid

All three passed the checklist on the first try, so the retry was never needed. But passing the checklist only proves the form is filled in correctly, not that the answers are true. So I also compared each result with the

original report by eye:

Report	Result	What I checked against the original
01 · Payments API	Valid · P1	Both times match (14:05 and 15:20). Impact and cause match. The two actions are the two in the report.
02 · ORD-ETL	Valid · P3	The incident is still open, so resolved time and root cause are empty . The model did not make up a time. The two actions match; nothing was added.
03 · Wi-Fi, 3rd floor	Valid · P2	The date 25/09/2026 was read as 25 September (not as a month 25). The ID was found on the first line. Actions match.

Mistakes found: no wrong values. There were two small things the extractor left out. In report 03 the system is just "Wi-Fi" and it lost "3rd floor, Bangalore office" (the schema has no field for the location). In report 02 the impact says the finance report was 90 minutes late, but does not say the job failed twice with a connection timeout.

Tests: proving the rules really work

I wrote tests that feed the schema bad data and check that it says no. These tests do not use the model or the internet, so they run instantly and are free. Each test starts with a correct report and breaks exactly one thing: a severity of "High", a resolved time before the start time, and an ID of "0917". The homework asked for four tests. I wrote six: I added one for an open incident (no resolved time) and one for the timezone rule.

```
tests/test_incident.py::test_a_valid_incident_passes PASSED [ 16%]
tests/test_incident.py::test_unknown_severity_is_rejected PASSED [ 33%]
tests/test_incident.py::test_resolved_before_started_is_rejected PASSED [ 50%]
tests/test_incident.py::test_badly_formed_incident_id_is_rejected PASSED [ 66%]
tests/test_incident.py::test_open_incident_needs_no_resolved_at_or_root_cause PASSED [ 83%]
tests/test_incident.py::test_mixed_timezone_is_rejected PASSED [100%]
===== 6 passed in 0.18s =====
```

Making a test fail on purpose. A test that has never failed has not proved anything. So I temporarily deleted the time-order rule. The matching test failed straight away ("DID NOT RAISE" means the bad data was wrongly accepted), and the other five still passed. I then put the rule back and checked with git that the file was identical to before.

```
> with pytest.raises(ValidationError, match="earlier than started_at"):
E Failed: DID NOT RAISE ValidationError

FAILED tests/test_incident.py::test_resolved_before_started_is_rejected
1 failed, 5 passed in 0.30s
```

The app: choosing the document type

The web page (app/streamlit_app.py) now has a **Document type** dropdown with Invoice and Incident. The choice decides which checklist is used, and the page messages say "invoice" or "incident" to match. The page remembers which type each result was extracted as, so changing the dropdown afterwards cannot relabel an old result. If a document still fails the checklist after the one retry, the page tells the user that a person should review it.

Review: the homework asks for the security-reviewer helper from Day 2, but it does not exist in this project or on this computer. So I ran Claude Code's built-in security review on the change instead. It found no problems. The dropdown only offers a fixed list of two options, and the results are shown using Streamlit's safe display.

New project rule: I added to CLAUDE.md: every extraction goes through a Pydantic schema, and anything still invalid after one retry is sent to a person, never corrected automatically.

Reflection

Next document type I would extract: IT support tickets from a helpdesk. Each one would become a form with the ticket number, the department of the person asking, the category, the priority and a short description of the problem.

The one rule the schema must enforce: the priority must be exactly one of the allowed levels (for example P1 to P4), never a free word like "urgent". If the priority is made up or missing, the ticket could go to the wrong team or be missed. So a ticket with an invalid priority must fail the check and be sent to a person, just as an invalid severity does for incidents.